6. TESTING

Testing is an important phase of project which verifies whether the developed or implement system satisfies the specified functional and non-functional requirements. and The purpose of testing is to identify errors, validate system functionality, check access permission like login register etc, and ensure that it CRUD operation is working correctly. Testing will be performed throughout development using a combination like unit testing, integration testing, system testing, and acceptance testing. All valid and invalid inputs will be tested to verify the correct, security, and reliability of the system. So because it will not crash when in production.

6.1 Acceptance Testing

Acceptance testing will verify that the Wildlife Information System meets the requirements defined in this Software Requirements Specification (SRS). The system will be considered acceptable when its major functions operate correctly and the specified acceptance criteria are satisfied. The acceptance criteria are as follows:

- Authorized users should be able to log in successfully and receive tokens like session and login token.
- The system should allow users which have permission to create, retrieve, update, and delete wildlife records.
- The system should validate Data which is newly added before storing it in the PostgreSQL database.
- Authorized users should be able to upload correctly formatted CSV files in which have animal movement information and other related to it.
- The system should validate uploaded CSV files and report or tell error about invalid or incomplete records appropriately.
- Users should be able to search using filter records and search by query, such as animal name and species and geographical information.
- The system should return correct HTTP status code from backend to check whether system is correct working or not. and meaningful error messages for invalid requests, unauthorized access, and records that do not exist. And the message will show in frontend UI.
- The system should have role based access control so that users can perform only the operations permitted by their assigned roles.
- Admin can create user account and assign appropriate roles. And can delete or update information. Basically can control All the system overflow.
- Users should be able to retrieve and export available wildlife information in the supported data formats.
- The system should prevent unauthorized users from accessing private API. And will add rate limiting for prevent brute force attack. and restricted operations.
- The system should store and retrieve correctly without any data loss or corruption or crashed software.

Acceptance testing will be performed before the final submission to confirm that the implemented system satisfies its specified requirements.

6.2 Test Cases

TC ID	Feature	Input	Expected Output
TC-01	User Login	Valid email and password	JWT token returned, status 200
TC-02	User Login (wrong)	Valid email, wrong password	Status 401 Unauthorized
TC-03	Add Animal	Valid animal data fields	Record saved, new ID returned
TC-04	Add Animal (missing)	Species name missing	Status 400, field error message
TC-05	Get Animal by ID	Valid animal ID in URL	Full animal record in JSON
TC-06	Get Animal (wrong ID)	ID that does not exist	Status 404 Not Found
TC-07	Search by Species	species=Snow Leopard query	List of matching animals
TC-08	CSV Upload (valid)	Correctly formatted CSV file	All rows saved successfully
TC-09	CSV Upload (bad)	CSV with missing coordinates	Rows skipped, error report shown
TC-10	API without a token	GET request without Auth header	Status 401 Unauthorized
TC-11	Export Data	Request to export search results	JSON or CSV file downloaded
TC-12	Admin creates user	New user details with role	User account created successfully

6.3 Test Plan

Testing Approach: Black Box Testing

The black box testing approach will be used as the main testing technique by the development team.

Tests are created solely based on input values and expected output results.

Black box testing is appropriate for this case due to the following reasons:

- There are specific inputs and expected outputs of API endpoints.
- Functional requirements will be validated without having to read the code.
- This reflects the way actual users and other applications will work with the product.

Test Levels

Unit Testing: Unit tests will be conducted for each API endpoint with the help of Postman. In each unit test, only one API endpoint is tested.

Integration Testing: Integration testing will be carried out on the completion of development of all the API endpoints, and they will be tested together.

System Testing: System testing will involve conducting tests for the entire application system from logging in to exporting the data via APIs.

Acceptance Testing: The final stage of testing in which involves testing against the requirements specification.

Test Tools

Postman: For manual API testing.

Browser DevTools: For testing the web interface.

Test Schedule

Phase	Test Type	When
After each endpoint	Unit Test (Postman)	During development
After full API build	Integration Test	After Increment 2
Before submission	System & Acceptance Test	Final week

Definitions & Acronyms

Term / Acronym	Definition
WIS	Wildlife Information System
API	Application Programmable Interface
REST	Representational State Transfer
JWT	JSON Web Token
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
CRUD	Create, Read, Update, Delete
CSV	Comma Separated Values
JSON	JavaScript Object Notation
DFD	Data Flow Diagram
ER	Entity Relationship